

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

MÔN: CÔNG NGHỆ PHẦN MỀM

XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ HỌC TẬP ỨNG DỤNG RAG

Giảng viên hướng dẫn: Thầy Nguyễn Văn Chiến

Nhóm thực hiện: Nhóm 9

1. Đào Văn Khánh - MSSV: 051206009944
2. Nguyễn Ninh Công Danh - MSSV: 079205006602
3. Trần Thị Kiều Trinh - MSSV: 066306009531
4. Nguyễn Hiền Đạt - MSSV: 079206026928
5. Lương Thành Đạt - MSSV: 056206003075
6. Phan Liên Hưng - MSSV: 034206016528
7. Phạm Hoàng Dũng - MSSV: 070205007005

Thành phố Hồ Chí Minh, 2026

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án “Xây dựng hệ thống Chatbot hỗ trợ hỏi đáp tài liệu bằng công nghệ RAG”, nhóm chúng em đã nhận được rất nhiều sự hướng dẫn, giúp đỡ và tạo điều kiện thuận lợi từ phía Nhà trường, Khoa Công nghệ Thông tin cùng quý Thầy, Cô.

Đầu tiên, nhóm xin gửi lời cảm ơn chân thành đến Ban Giám hiệu Nhà trường cùng toàn thể quý Thầy, Cô thuộc Khoa Công nghệ Thông tin đã truyền đạt những kiến thức nền tảng cũng như những kinh nghiệm quý báu trong suốt quá trình học tập. Đây chính là tiền đề quan trọng giúp nhóm có đủ kiến thức để hoàn thành đồ án.

Đặc biệt, nhóm xin gửi lời cảm ơn sâu sắc đến Giảng viên hướng dẫn đã tận tình hướng dẫn, góp ý và hỗ trợ nhóm trong suốt quá trình nghiên cứu, phân tích yêu cầu, thiết kế hệ thống, triển khai chương trình cũng như hoàn thiện báo cáo. Những góp ý của Thầy/Cô đã giúp nhóm hoàn thiện sản phẩm cả về mặt kỹ thuật lẫn nội dung báo cáo.

Trong quá trình thực hiện, nhóm đã tìm hiểu và ứng dụng nhiều công nghệ hiện đại như Retrieval-Augmented Generation (RAG), ChromaDB, FastAPI, ReactJS, mô hình ngôn ngữ lớn (LLM), kỹ thuật Embedding văn bản và xây dựng API phục vụ hệ thống Chatbot. Thông qua quá trình này, nhóm không chỉ củng cố kiến thức chuyên môn mà còn rèn luyện kỹ năng nghiên cứu tài liệu, làm việc nhóm, quản lý tiến độ bằng Jira và quản lý mã nguồn bằng GitHub.

Mặc dù nhóm đã cố gắng hoàn thành đồ án một cách tốt nhất, tuy nhiên do thời gian thực hiện và kinh nghiệm thực tế còn hạn chế nên báo cáo chắc chắn vẫn còn nhiều thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ quý Thầy, Cô để có thể tiếp tục hoàn thiện sản phẩm trong tương lai.

Cuối cùng, nhóm xin kính chúc quý Thầy, Cô luôn dồi dào sức khỏe, hạnh phúc và thành công trong sự nghiệp giảng dạy.

Xin chân thành cảm ơn!

Chapter 1

GIỚI THIỆU

1.1 Giới thiệu đề tài

Trong bối cảnh trí tuệ nhân tạo (Artificial Intelligence - AI) phát triển mạnh mẽ, các mô hình ngôn ngữ lớn (Large Language Models - LLM) đang được ứng dụng rộng rãi trong nhiều lĩnh vực như giáo dục, y tế, tài chính và dịch vụ khách hàng. Đặc biệt, các hệ thống Chatbot thông minh đã trở thành công cụ hỗ trợ hiệu quả trong việc tìm kiếm, tổng hợp và trả lời thông tin.

Trong lĩnh vực giáo dục, sinh viên thường gặp khó khăn khi phải tìm kiếm thông tin từ nhiều nguồn tài liệu khác nhau như giáo trình, slide bài giảng, tài liệu PDF hoặc tài liệu tham khảo. Việc đọc và tổng hợp lượng lớn dữ liệu mất nhiều thời gian và đôi khi không mang lại kết quả chính xác.

Xuất phát từ nhu cầu đó, nhóm thực hiện đề tài:

XÂY DỰNG HỆ THỐNG CHATBOT HỖ TRỢ HỌC TẬP ỨNG DỤNG RETRIEVAL-AUGMENTED GENERATION (RAG)

Hệ thống được xây dựng với mục tiêu hỗ trợ sinh viên đặt câu hỏi trực tiếp về nội dung môn học. Chatbot sẽ truy xuất thông tin từ tài liệu đã được tải lên, kết hợp cùng mô hình ngôn ngữ lớn để tạo ra câu trả lời có độ chính xác cao và sát với tài liệu gốc.

Bên cạnh đó, đề tài còn nghiên cứu, đánh giá và so sánh hiệu quả giữa hai phương pháp phổ biến hiện nay là Retrieval-Augmented Generation (RAG) và Fine-Tuning nhằm xác định giải pháp phù hợp cho bài toán hỏi đáp tài liệu học tập.

1.2 Lý do chọn đề tài

Hiện nay, phần lớn sinh viên sử dụng các công cụ AI như ChatGPT hoặc Gemini để hỗ trợ học tập. Tuy nhiên, các mô hình này chủ yếu dựa trên dữ liệu được huấn luyện trước và không thể truy cập trực tiếp vào tài liệu nội bộ của từng môn học nếu không được cung cấp.

Điều này dẫn đến một số hạn chế:

- Câu trả lời có thể không đúng với giáo trình của môn học.
- Khó trích dẫn nguồn tài liệu.
- Không thể cập nhật nhanh khi tài liệu môn học thay đổi.
- Độ chính xác giảm đối với các kiến thức chuyên ngành.

Trong khi đó, kỹ thuật Retrieval-Augmented Generation (RAG) cho phép hệ thống truy xuất trực tiếp thông tin từ các tài liệu do người dùng cung cấp trước khi sinh câu trả lời. Đồng thời, Fine-Tuning giúp mô hình học sâu hơn trên một tập dữ liệu chuyên biệt để nâng cao chất lượng phản hồi.

Vì vậy, nhóm lựa chọn nghiên cứu và xây dựng hệ thống Chatbot kết hợp RAG và Fine-Tuning nhằm hỗ trợ học tập, đồng thời đánh giá ưu nhược điểm của từng phương pháp trong thực tế.

1.3 Mục tiêu đề tài

Đề tài hướng đến các mục tiêu sau:

- Xây dựng hệ thống Chatbot hỗ trợ trả lời câu hỏi liên quan đến tài liệu môn học.
- Cho phép người quản trị tải lên và quản lý tài liệu PDF phục vụ việc hỏi đáp.
- Xây dựng quy trình xử lý tài liệu bao gồm tách văn bản, chia nhỏ dữ liệu (Chunking), tạo Embedding và lưu trữ trong cơ sở dữ liệu vector ChromaDB.
- Tích hợp mô hình ngôn ngữ lớn thông qua FastAPI để sinh câu trả lời.
- Xây dựng giao diện Web bằng React giúp người dùng dễ dàng tương tác với hệ thống.
- So sánh chất lượng giữa mô hình RAG và Fine-Tuning dựa trên các tiêu chí như độ chính xác, tốc độ phản hồi và khả năng bám sát tài liệu.
- Đánh giá khả năng ứng dụng của hệ thống trong môi trường học tập thực tế.

1.4 Phạm vi nghiên cứu

Đề tài tập trung nghiên cứu và triển khai các nội dung sau:

- Xây dựng Chatbot phục vụ lĩnh vực học tập.
- Hỗ trợ truy vấn thông tin từ tài liệu PDF.
- Triển khai kỹ thuật RAG kết hợp ChromaDB.
- Nghiên cứu Fine-Tuning trên tập dữ liệu môn học.
- Phát triển Backend bằng FastAPI.
- Phát triển Frontend bằng ReactJS.
- Đánh giá hiệu năng và độ chính xác giữa RAG và Fine-Tuning.

Đề tài chưa nghiên cứu các nội dung như nhận dạng giọng nói, xử lý hình ảnh, đa ngôn ngữ hoặc triển khai trên môi trường Cloud quy mô lớn.

1.5 Đối tượng sử dụng

Hệ thống được xây dựng dành cho hai nhóm người dùng chính:

Sinh viên

- Đặt câu hỏi về nội dung môn học.
- Tra cứu thông tin nhanh.
- So sánh kết quả giữa RAG và Fine-Tuning.
- Xem báo cáo đánh giá chất lượng câu trả lời.

Quản trị viên

- Quản lý tài liệu môn học.
- Cập nhật nguồn dữ liệu.
- Theo dõi trạng thái hệ thống.
- Xem báo cáo thống kê.
- Quản lý quá trình Fine-Tuning.

1.6 Phương pháp nghiên cứu

Đề tài được thực hiện theo các bước:

1. Khảo sát các mô hình Chatbot hiện nay.
2. Nghiên cứu kiến trúc RAG.
3. Nghiên cứu phương pháp Fine-Tuning.
4. Thiết kế kiến trúc hệ thống.
5. Xây dựng Backend bằng FastAPI.
6. Xây dựng Frontend bằng React.
7. Tích hợp ChromaDB và Embedding.
8. Kiểm thử chức năng.
9. Đánh giá kết quả.

1.7 Cấu trúc báo cáo

Báo cáo gồm các chương sau:

- Chương 1: Giới thiệu.
- Chương 2: Cơ sở lý thuyết và công nghệ sử dụng.
- Chương 3: Đặc tả yêu cầu phần mềm (SRS).
- Chương 4: Thiết kế hệ thống.
- Chương 5: Triển khai hệ thống.
- Chương 6: Kiểm thử và đánh giá.
- Chương 7: Quản lý dự án bằng Jira và GitHub.
- Chương 8: Kết luận và hướng phát triển.

Chapter 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1 Trí tuệ nhân tạo (Artificial Intelligence)

Trí tuệ nhân tạo (Artificial Intelligence - AI) là lĩnh vực nghiên cứu thuộc khoa học máy tính nhằm xây dựng các hệ thống có khả năng mô phỏng trí thông minh của con người. AI cho phép máy tính thực hiện nhiều nhiệm vụ như học tập, suy luận, lập kế hoạch, xử lý ngôn ngữ tự nhiên, nhận dạng hình ảnh và đưa ra quyết định.

Trong những năm gần đây, AI phát triển mạnh nhờ sự gia tăng về năng lực tính toán, dữ liệu lớn (Big Data) và các mô hình học sâu (Deep Learning). AI đã được ứng dụng rộng rãi trong nhiều lĩnh vực như giáo dục, tài chính, thương mại điện tử, y tế, giao thông và dịch vụ khách hàng.

Trong đề tài này, AI đóng vai trò là nền tảng giúp hệ thống Chatbot hiểu câu hỏi của người dùng, truy xuất thông tin từ tài liệu và sinh ra câu trả lời phù hợp.

2.2 Mô hình ngôn ngữ lớn (Large Language Model)

Large Language Model (LLM) là các mô hình học sâu được huấn luyện trên tập dữ liệu văn bản rất lớn nhằm hiểu và sinh ngôn ngữ tự nhiên.

LLM có khả năng:

- Trả lời câu hỏi.
- Tóm tắt văn bản.
- Dịch ngôn ngữ.
- Sinh mã nguồn.

- Phân tích tài liệu.
- Hội thoại với người dùng.

Các mô hình phổ biến hiện nay gồm GPT, Llama, Gemini, Claude, Qwen,...
 Trong dự án, LLM chịu trách nhiệm tạo ra câu trả lời cuối cùng sau khi nhận được dữ liệu liên quan từ hệ thống Retrieval.

2.3 Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) là kỹ thuật kết hợp giữa truy xuất dữ liệu (Retrieval) và mô hình sinh ngôn ngữ (Generation).

Thay vì chỉ dựa vào kiến thức đã được huấn luyện trước, hệ thống sẽ tìm kiếm các đoạn tài liệu liên quan trước khi gửi vào mô hình AI.

Quy trình hoạt động gồm các bước:

1. Người dùng gửi câu hỏi.
2. Câu hỏi được chuyển thành Vector Embedding.
3. Hệ thống tìm kiếm các đoạn văn bản tương đồng trong ChromaDB.
4. Các đoạn tài liệu được đưa vào Prompt.
5. LLM sinh câu trả lời dựa trên tài liệu vừa truy xuất.

Nhờ đó, câu trả lời luôn bám sát tài liệu môn học và giảm hiện tượng Hallucination.

2.4 Fine-Tuning

Fine-Tuning là quá trình huấn luyện tiếp một mô hình ngôn ngữ đã được huấn luyện trước bằng tập dữ liệu chuyên biệt.

Mục tiêu của Fine-Tuning là giúp mô hình:

- Hiểu ngữ cảnh chuyên ngành.
- Trả lời đúng theo dữ liệu huấn luyện.
- Tăng độ chính xác.
- Giảm câu trả lời sai.

Trong đề tài, nhóm tiến hành nghiên cứu Fine-Tuning trên dữ liệu môn học nhằm so sánh hiệu quả với phương pháp RAG.

2.5 Embedding

Embedding là kỹ thuật chuyển đổi văn bản thành vector số có nhiều chiều.

Các vector này phản ánh ý nghĩa ngữ nghĩa của văn bản thay vì chỉ dựa vào từ khóa.

Quá trình Embedding trong hệ thống bao gồm:

1. Đọc tài liệu PDF.
2. Chia nhỏ thành nhiều đoạn (Chunk).
3. Sinh Vector Embedding.
4. Lưu Vector vào ChromaDB.
5. Khi người dùng hỏi, câu hỏi cũng được chuyển thành Vector.
6. So sánh độ tương đồng Cosine Similarity.

Embedding giúp tăng tốc độ truy xuất và cải thiện độ chính xác của hệ thống.

2.6 ChromaDB

ChromaDB là cơ sở dữ liệu Vector mã nguồn mở dùng để lưu trữ Embedding.

Khác với cơ sở dữ liệu truyền thống chỉ lưu dữ liệu dạng bảng, ChromaDB lưu các Vector có hàng trăm hoặc hàng nghìn chiều.

Trong hệ thống, ChromaDB thực hiện:

- Lưu trữ Embedding.
- Tìm kiếm Vector gần nhất.
- Truy xuất các đoạn tài liệu liên quan.
- Trả về dữ liệu cho mô hình AI.

Việc sử dụng ChromaDB giúp hệ thống tìm kiếm dữ liệu nhanh và chính xác ngay cả khi số lượng tài liệu lớn.

2.7 FastAPI

FastAPI là Framework phát triển REST API bằng ngôn ngữ Python.

Các ưu điểm của FastAPI gồm:

- Hiệu năng cao.
- Dễ xây dựng API.

- Tự sinh Swagger Documentation.
- Hỗ trợ Async.
- Dễ tích hợp AI.

Trong đề tài, FastAPI đảm nhiệm:

- Nhận câu hỏi từ Frontend.
- Gọi ChromaDB.
- Gọi mô hình AI.
- Trả kết quả cho React.
- Quản lý Upload PDF.

2.8 ReactJS

ReactJS là thư viện JavaScript được Meta phát triển nhằm xây dựng giao diện người dùng theo mô hình Component.

React giúp:

- Giao diện hiện đại.
- Cập nhật dữ liệu theo thời gian thực.
- Dễ mở rộng.
- Tái sử dụng Component.

Trong hệ thống, ReactJS được sử dụng để xây dựng giao diện Chatbot, quản lý tài liệu và hiển thị kết quả so sánh giữa RAG và Fine-Tuning.

2.9 Kiến trúc tổng thể hệ thống

Hình 3.1 mô tả kiến trúc tổng thể của hệ thống Chatbot hỗ trợ học tập.

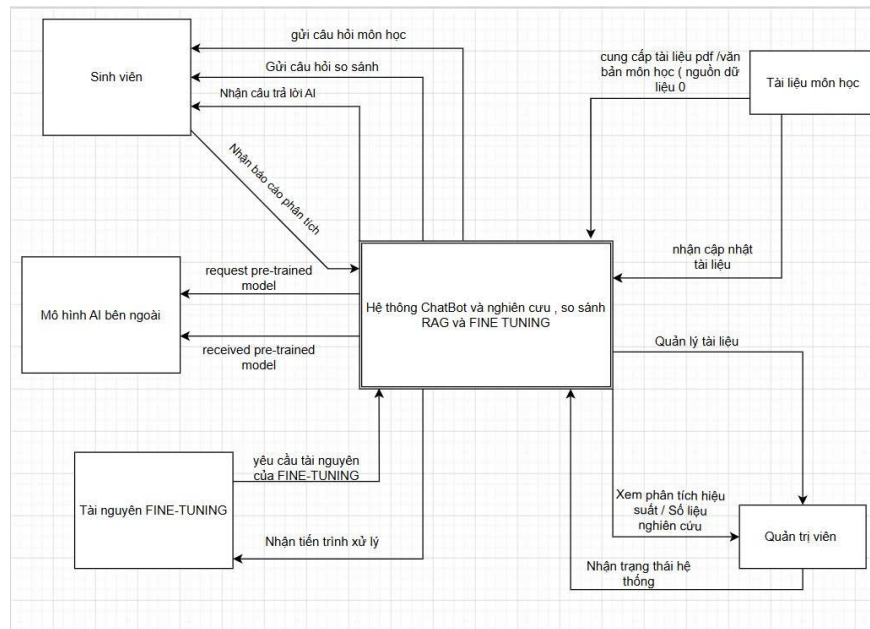


Figure 2.1: Kiến trúc tổng thể hệ thống

Kiến trúc hệ thống gồm năm thành phần chính:

- Người dùng (Sinh viên).
- Quản trị viên.
- Hệ thống Chatbot.
- Kho tài liệu môn học.
- Mô hình AI và tài nguyên Fine-Tuning.

Sinh viên gửi câu hỏi đến hệ thống thông qua giao diện Web. Hệ thống thực hiện truy xuất dữ liệu trong kho tài liệu bằng kỹ thuật RAG, sau đó kết hợp với mô hình AI để sinh câu trả lời.

Quản trị viên chịu trách nhiệm cập nhật tài liệu, theo dõi trạng thái hệ thống, đánh giá hiệu suất và quản lý quá trình Fine-Tuning.

Nhờ kiến trúc này, dữ liệu được quản lý tập trung, dễ mở rộng và thuận tiện trong việc cập nhật tài liệu môn học.

Chapter 3

ĐẶC TẢ YÊU CẦU PHẦN MỀM (SRS)

3.1 Giới thiệu

Tài liệu Đặc tả yêu cầu phần mềm (Software Requirements Specification - SRS) mô tả đầy đủ các yêu cầu của hệ thống Chatbot hỗ trợ học tập ứng dụng Retrieval-Augmented Generation (RAG) và Fine-Tuning. Tài liệu đóng vai trò là cơ sở để nhóm phát triển tiến hành thiết kế, triển khai, kiểm thử và đánh giá hệ thống.

Các yêu cầu được xây dựng dựa trên nhu cầu thực tế của sinh viên trong việc tra cứu tài liệu học tập, đồng thời hỗ trợ nghiên cứu và so sánh hiệu quả giữa hai phương pháp RAG và Fine-Tuning.

3.2 Mục đích

Mục đích của tài liệu SRS là:

- Xác định đầy đủ các chức năng mà hệ thống cần cung cấp.
- Xác định các yêu cầu phi chức năng.
- Làm cơ sở cho việc thiết kế hệ thống.
- Hỗ trợ kiểm thử và nghiệm thu sản phẩm.
- Giảm sai lệch giữa yêu cầu và quá trình phát triển.

3.3 Phạm vi hệ thống

Hệ thống Chatbot được xây dựng nhằm hỗ trợ sinh viên đặt câu hỏi liên quan đến học tập và nhận câu trả lời từ mô hình AI.

Hệ thống cung cấp hai phương pháp xử lý:

- Chatbot sử dụng Retrieval-Augmented Generation (RAG).
- Chatbot sử dụng mô hình Fine-Tuning.

Ngoài ra hệ thống còn cho phép quản trị viên quản lý tài liệu, theo dõi quá trình Fine-Tuning, đánh giá hiệu quả mô hình và xuất báo cáo thống kê.

3.4 Các tác nhân của hệ thống

Hệ thống gồm các tác nhân chính:

- Sinh viên
- Quản trị viên
- Mô hình AI
- Kho tài liệu môn học
- Hệ thống Fine-Tuning

3.5 Biểu đồ ngữ cảnh (Context Diagram)

Biểu đồ Context mô tả hệ thống ở mức tổng quát nhất, thể hiện các tác nhân bên ngoài và luồng dữ liệu trao đổi với hệ thống.

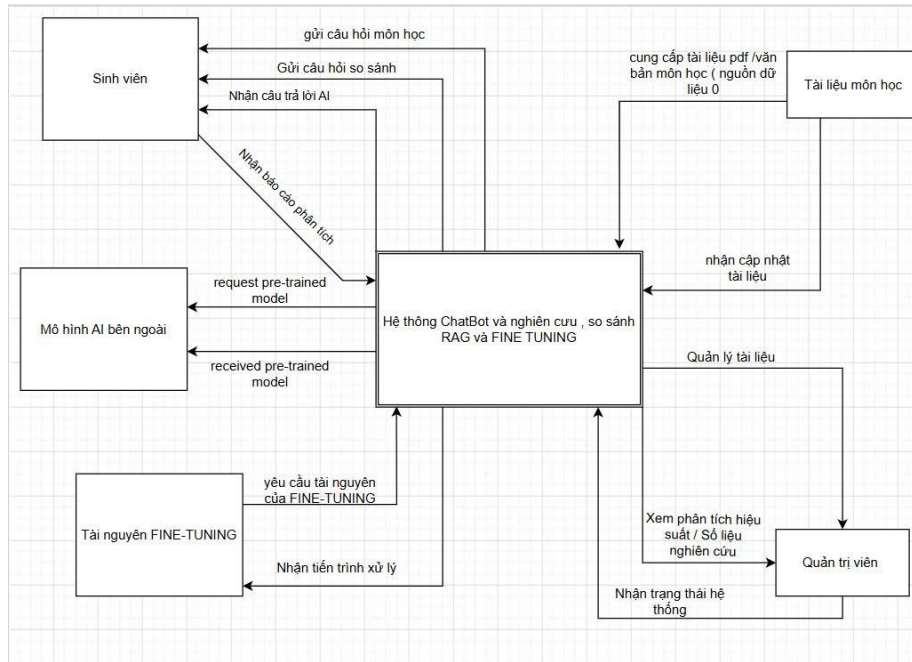


Figure 3.1: Biểu đồ Context của hệ thống

Trong biểu đồ Context, hệ thống Chatbot đóng vai trò trung tâm tiếp nhận và xử lý mọi yêu cầu từ các tác nhân bên ngoài.

Sinh viên gửi câu hỏi về môn học hoặc yêu cầu so sánh giữa hai phương pháp RAG và Fine-Tuning. Sau khi xử lý, hệ thống trả về câu trả lời cùng các báo cáo phân tích nếu người dùng yêu cầu.

Kho tài liệu môn học cung cấp dữ liệu PDF, Word và các tài liệu học tập để phục vụ quá trình truy xuất thông tin của RAG.

Mô hình AI bên ngoài được sử dụng để sinh câu trả lời thông minh dựa trên Prompt do hệ thống tạo ra.

Tài nguyên Fine-Tuning chịu trách nhiệm cung cấp môi trường huấn luyện và cập nhật mô hình mới.

Quản trị viên có quyền quản lý tài liệu, theo dõi trạng thái hệ thống, xem thống kê và đánh giá hiệu quả của các mô hình AI.

3.6 Biểu đồ Use Case

Biểu đồ Use Case mô tả toàn bộ các chức năng mà hệ thống cung cấp cho từng tác nhân.

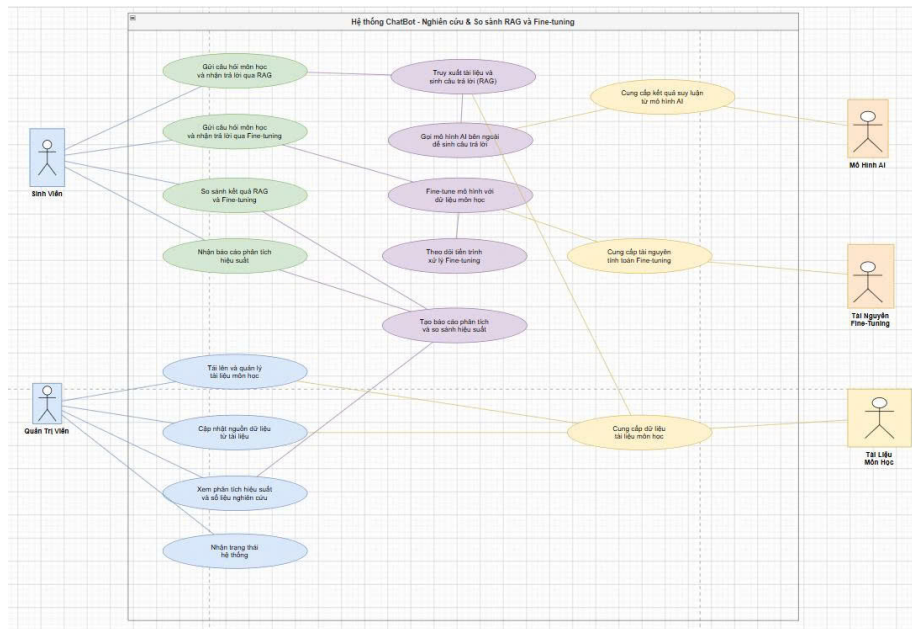


Figure 3.2: Biểu đồ Use Case

Biểu đồ Use Case cho thấy hệ thống gồm hai nhóm người dùng chính là Sinh viên và Quản trị viên.

Sinh viên có thể:

- Gửi câu hỏi bằng RAG.
- Gửi câu hỏi bằng Fine-Tuning.
- So sánh kết quả giữa hai phương pháp.
- Xem báo cáo đánh giá.

Quản trị viên có thể:

- Tải lên tài liệu mới.
- Quản lý kho dữ liệu.
- Cập nhật tài liệu.
- Theo dõi trạng thái hệ thống.
- Xem báo cáo phân tích.
- Quản lý quá trình Fine-Tuning.

Trong quá trình xử lý, hệ thống sẽ gọi tới mô hình AI, truy xuất dữ liệu từ ChromaDB theo cơ chế RAG hoặc sử dụng mô hình Fine-Tuning để sinh câu trả lời.

3.7 Yêu cầu chức năng

Hệ thống cần đáp ứng các yêu cầu chức năng sau:

1. Cho phép sinh viên nhập câu hỏi.
2. Truy xuất dữ liệu từ ChromaDB.
3. Sinh câu trả lời bằng RAG.
4. Sinh câu trả lời bằng Fine-Tuning.
5. So sánh kết quả giữa hai phương pháp.
6. Lưu lịch sử hội thoại.
7. Quản lý tài liệu học tập.
8. Quản lý người dùng.
9. Sinh báo cáo thống kê.
10. Theo dõi tiến trình Fine-Tuning.

3.8 Yêu cầu phi chức năng

1. **Hiệu năng:** Thời gian phản hồi trung bình nhỏ hơn 5 giây đối với câu hỏi thông thường.
2. **Độ chính xác:** Hệ thống ưu tiên trả lời dựa trên tài liệu môn học trước khi sử dụng tri thức tổng quát của mô hình AI.
3. **Khả năng mở rộng:** Có thể bổ sung tài liệu mới mà không cần thay đổi chương trình.
4. **Bảo mật:** Chỉ quản trị viên mới được phép cập nhật tài liệu và quản lý hệ thống.
5. **Khả dụng :** Hệ thống hoạt động liên tục và hỗ trợ nhiều người dùng đồng thời thông qua FastAPI.

3.9 Kết luận chương

Chương này đã trình bày đầy đủ đặc tả yêu cầu phần mềm của hệ thống Chatbot hỗ trợ học tập, bao gồm mục đích, phạm vi, các yêu cầu chức năng, yêu cầu phi chức năng cùng hai sơ đồ quan trọng là Context Diagram và Use Case Diagram. Đây là cơ sở để tiến hành thiết kế kiến trúc và triển khai hệ thống trong các chương tiếp theo.

Chapter 4

THIẾT KẾ HỆ THỐNG

4.1 Giới thiệu

Sau khi xác định đầy đủ các yêu cầu trong tài liệu SRS, chương này trình bày quá trình thiết kế hệ thống Chatbot hỗ trợ học tập ứng dụng Retrieval-Augmented Generation (RAG) và Fine-Tuning. Nội dung bao gồm thiết kế kiến trúc phần mềm, thiết kế các lớp của hệ thống, thiết kế cơ sở dữ liệu và mô tả luồng xử lý giữa các thành phần.

Việc thiết kế hướng đến mục tiêu đảm bảo hệ thống có khả năng mở rộng, dễ bảo trì, dễ tích hợp thêm mô hình AI mới cũng như đáp ứng tốt yêu cầu nghiên cứu và so sánh giữa RAG và Fine-Tuning.

4.2 Kiến trúc tổng thể

Hệ thống được xây dựng theo mô hình Client – Server kết hợp kiến trúc AI Pipeline.

Các thành phần chính bao gồm:

- Frontend ReactJS
- Backend FastAPI
- ChromaDB
- LLM (Gemini/OpenAI)
- Module Fine-Tuning
- Cơ sở dữ liệu MySQL

Quy trình hoạt động gồm các bước:

1. Người dùng gửi câu hỏi.

- Backend kiểm tra phương pháp xử lý.
- Nếu chọn RAG, hệ thống truy xuất tài liệu trong ChromaDB.
- Prompt được xây dựng từ dữ liệu truy xuất.
- LLM sinh câu trả lời.
- Nếu chọn Fine-Tuning, câu hỏi được gửi trực tiếp tới mô hình đã huấn luyện.
- Kết quả trả về giao diện React.

4.3 Thiết kế lớp (Class Diagram)

Biểu đồ lớp mô tả cấu trúc hướng đối tượng của toàn bộ hệ thống.

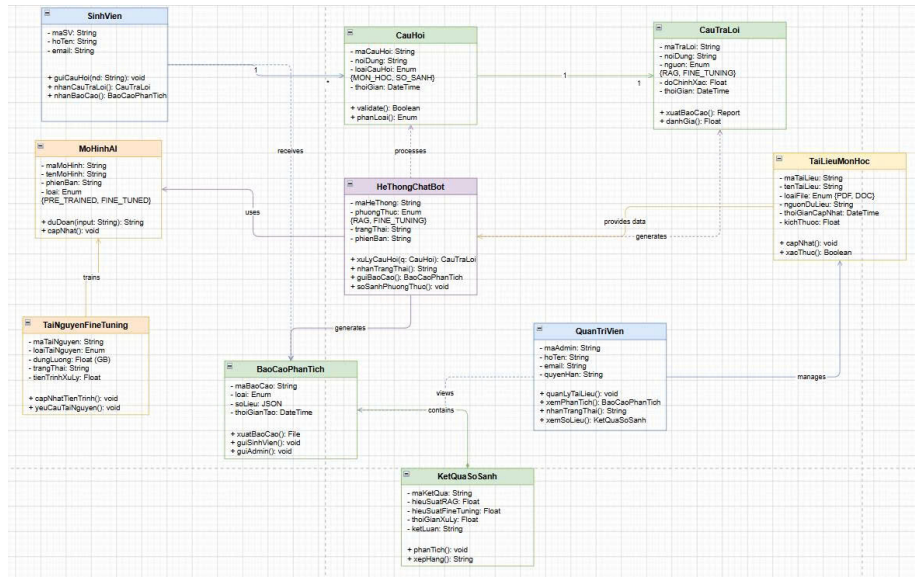


Figure 4.1: Biểu đồ lớp của hệ thống

Biểu đồ lớp gồm các lớp chính sau:

SinhVien

Đại diện cho người sử dụng hệ thống.

Các chức năng:

- Gửi câu hỏi.
- Nhận câu trả lời.

- Nhận báo cáo đánh giá.

Sinh viên là tác nhân trực tiếp tương tác với hệ thống Chatbot.

CauHoi

Lưu toàn bộ thông tin câu hỏi do người dùng gửi lên.

Bao gồm:

- Nội dung câu hỏi.
- Loại câu hỏi.
- Môn học.
- Thời gian gửi.

Sau khi tiếp nhận, câu hỏi sẽ được chuyển tới lớp HệThốngChatbot để xử lý.

HeThongChatbot

Đây là lớp trung tâm của hệ thống.

Các nhiệm vụ chính:

- Điều phối quá trình xử lý.
- Chọn RAG hoặc Fine-Tuning.
- Gọi mô hình AI.
- Sinh báo cáo.
- So sánh hai phương pháp.

Mọi thành phần đều kết nối thông qua lớp này.

MoHinhAI

Đại diện cho các mô hình AI bên ngoài.

Ví dụ:

- Gemini
- GPT
- Llama

Hệ thống gửi Prompt và nhận câu trả lời từ mô hình AI.

TaiLieuMonHoc

Quản lý toàn bộ tài liệu PDF, DOCX và TXT.

Tài liệu sau khi tải lên sẽ được chia nhỏ (Chunking) và lưu vào ChromaDB để phục vụ RAG.

TaiNguyenFineTuning

Lưu các tài nguyên phục vụ Fine-Tuning như:

- Dataset
- GPU
- Dung lượng
- Thời gian xử lý

BaoCaoPhanTich

Lưu toàn bộ kết quả đánh giá.

Bao gồm:

- Accuracy
- Thời gian phản hồi
- Chi phí
- Kết luận

KetQuaSoSanh

Thực hiện so sánh giữa RAG và Fine-Tuning dựa trên nhiều tiêu chí khác nhau.

QuanTriVien

Quản lý:

- Người dùng
- Tài liệu
- Báo cáo
- Trạng thái hệ thống

4.4 Thiết kế cơ sở dữ liệu

Hệ thống sử dụng cơ sở dữ liệu quan hệ để quản lý thông tin người dùng, tài liệu, câu hỏi và báo cáo đánh giá.

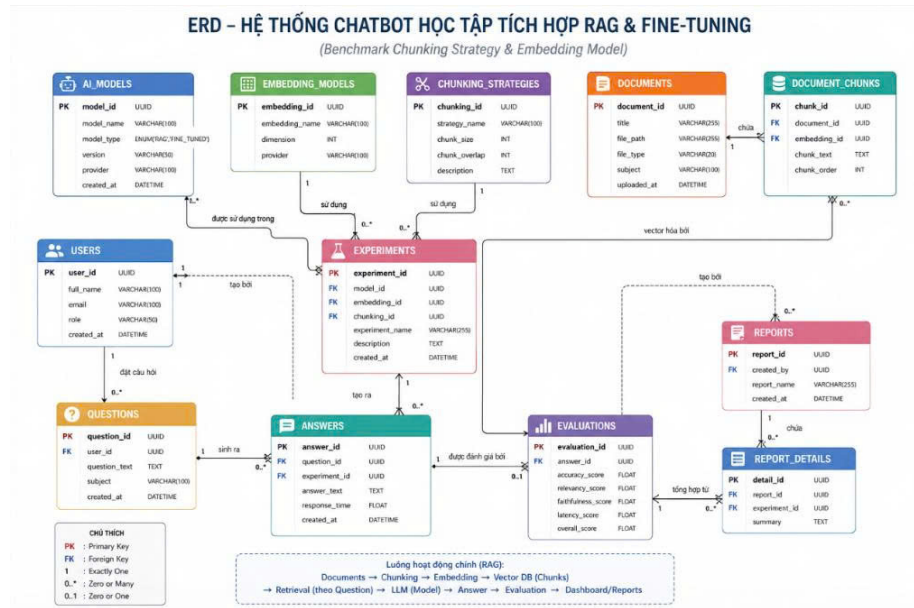


Figure 4.2: ERD của hệ thống

ERD mô tả toàn bộ các bảng dữ liệu phục vụ hệ thống.

Users

Lưu thông tin sinh viên và quản trị viên.

Questions

Lưu lịch sử câu hỏi của người dùng.

Mỗi câu hỏi thuộc về một người dùng.

Answers

Lưu câu trả lời do AI sinh ra.

Mỗi câu trả lời gắn với một câu hỏi.

Documents

Lưu thông tin tài liệu gốc.

Bao gồm:

- Tên tài liệu
- Đường dẫn
- Môn học
- Ngày tải lên

Document Chunks

Lưu các đoạn văn sau khi Chunking.

Đây là dữ liệu được đưa vào ChromaDB.

Embedding Models

Quản lý các mô hình Embedding.

Ví dụ:

- BAAI
- Sentence Transformer
- OpenAI Embedding

Chunking Strategies

Lưu chiến lược chia nhỏ tài liệu.

Ví dụ:

- Fixed Size
- Recursive
- Semantic Chunking

Experiments

Lưu từng lần chạy thử nghiệm giữa RAG và Fine-Tuning.

Evaluations

Lưu các chỉ số đánh giá:

- Accuracy
- Relevancy
- Latency
- Faithfulness

Reports

Lưu báo cáo tổng hợp cuối cùng của từng lần thử nghiệm.

4.5 Luồng xử lý dữ liệu

Khi người dùng gửi câu hỏi, hệ thống sẽ thực hiện theo các bước sau:

1. Tiếp nhận câu hỏi từ giao diện React.
2. FastAPI nhận Request.
3. Kiểm tra chế độ RAG hoặc Fine-Tuning.
4. Nếu là RAG:
 - Chunking tài liệu.
 - Embedding.
 - Truy vấn ChromaDB.
 - Sinh Prompt.
 - Gửi tới LLM.
5. Nếu là Fine-Tuning:
 - Gửi trực tiếp tới mô hình đã huấn luyện.
6. Nhận câu trả lời.
7. Ghi lịch sử.
8. Trả kết quả về giao diện.

4.6 Kết luận chương

Chương này đã trình bày chi tiết thiết kế hệ thống Chatbot từ góc nhìn hướng đối tượng và cơ sở dữ liệu. Class Diagram mô tả các lớp và mối quan hệ giữa các thành phần, trong khi ERD thể hiện cấu trúc dữ liệu phục vụ cho quá trình lưu trữ và xử lý. Những thiết kế này là nền tảng để triển khai hệ thống ở chương tiếp theo.

Chapter 5

Triển khai hệ thống

5.1 Môi trường triển khai

Hệ thống ChatBot được triển khai theo mô hình Client–Server. Phần giao diện người dùng được xây dựng bằng ReactJS, Backend sử dụng FastAPI và toàn bộ dữ liệu vector được lưu trên ChromaDB. Hệ thống AI sử dụng mô hình Gemini 2.5 Flash kết hợp với cơ chế Retrieval-Augmented Generation (RAG).

Trong quá trình triển khai, hệ thống được chia thành các module riêng biệt gồm:

- Giao diện người dùng (Frontend)
- API xử lý nghiệp vụ (FastAPI)
- Module RAG
- ChromaDB
- Module Benchmark
- Dashboard thống kê

5.2 Chức năng Chat Sandbox

Chat Sandbox là chức năng chính của hệ thống, cho phép sinh viên đặt câu hỏi trực tiếp dựa trên các tài liệu đã được tải lên.

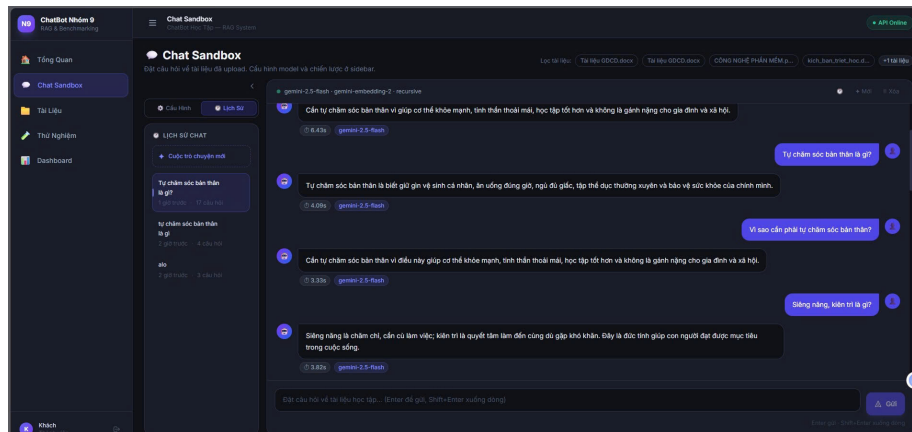


Figure 5.1: Giao diện Chat Sandbox

Sau khi người dùng nhập câu hỏi, hệ thống sẽ gửi yêu cầu đến FastAPI. Backend tiến hành:

1. Nhận câu hỏi
2. Thực hiện embedding
3. Truy vấn ChromaDB
4. Lấy các đoạn tài liệu liên quan
5. Tạo Prompt hoàn chỉnh
6. Gửi sang Gemini 2.5 Flash
7. Nhận kết quả
8. Hiện thị lên giao diện

Ngoài nội dung trả lời, hệ thống còn hiển thị:

- Model AI sử dụng
- Thời gian phản hồi
- Lịch sử hội thoại
- Danh sách tài liệu đang được sử dụng

Việc lưu lịch sử hội thoại giúp người dùng dễ dàng tiếp tục trao đổi mà không cần nhập lại các câu hỏi trước đó.

5.3 Chức năng quản lý tài liệu

Hệ thống cho phép quản trị viên tải lên các tài liệu PDF hoặc DOCX để xây dựng kho tri thức phục vụ quá trình Retrieval.

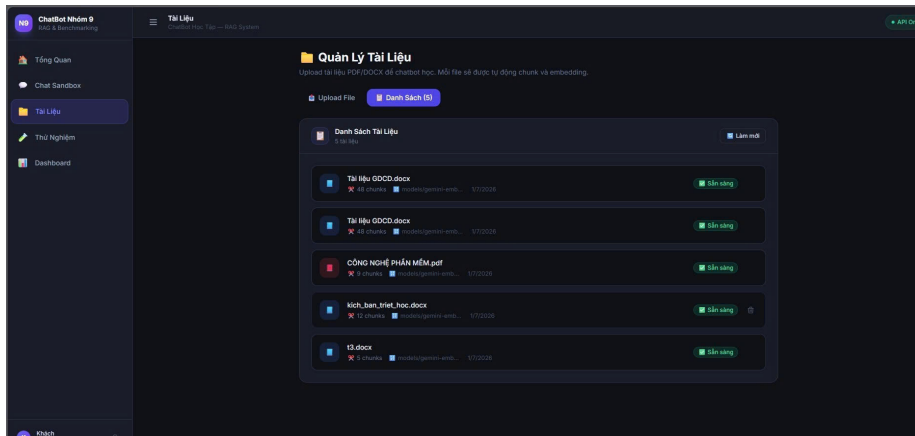


Figure 5.2: Chức năng quản lý tài liệu

Sau khi tải lên, hệ thống tự động thực hiện:

- Đọc nội dung tài liệu
- Chia tài liệu thành nhiều đoạn (Chunking)
- Sinh Vector Embedding
- Lưu vào ChromaDB
- Cập nhật trạng thái tài liệu

Danh sách tài liệu sẽ hiển thị:

- Tên tài liệu
- Số lượng Chunk
- Mô hình Embedding
- Ngày tải lên
- Trạng thái xử lý

Khi trạng thái chuyển sang "Sẵn sàng", tài liệu có thể được sử dụng để trả lời câu hỏi trong Chat Sandbox.

5.4 Luồng xử lý của hệ thống RAG

Quá trình xử lý của hệ thống được thực hiện theo các bước:

1. Người dùng nhập câu hỏi.
2. FastAPI nhận request.
3. Sinh vector embedding cho câu hỏi.
4. Truy vấn ChromaDB.
5. Lấy Top-K đoạn tài liệu phù hợp.
6. Ghép Prompt.
7. Gửi Prompt sang Gemini.
8. Gemini sinh câu trả lời.
9. Hiển thị kết quả lên giao diện.

Quy trình trên giúp hạn chế hiện tượng AI trả lời ngoài phạm vi dữ liệu và nâng cao độ chính xác của câu trả lời.

Chapter 6

Kiểm thử và đánh giá hệ thống

6.1 Kiểm thử chức năng

Nhóm tiến hành kiểm thử toàn bộ các chức năng chính của hệ thống.

Các chức năng được kiểm thử gồm:

- Upload tài liệu
- Chat Sandbox
- Benchmark
- Dashboard
- Quản lý dữ liệu

Kết quả cho thấy các chức năng đều hoạt động ổn định và đáp ứng đúng yêu cầu đặt ra.

6.2 Thử nghiệm Benchmark

Để đánh giá hiệu năng của hệ thống, nhóm sử dụng module Benchmark.

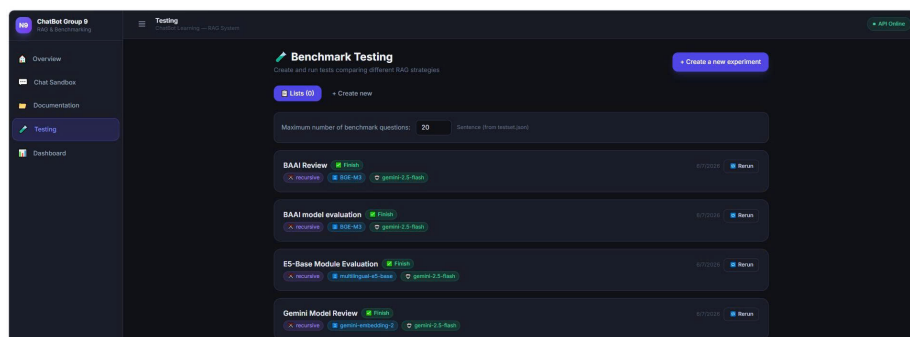


Figure 6.1: Danh Sách Các Thử Nghiệm (Testing Benchmark)

Quy mô thử nghiệm: Hình ảnh cho thấy hệ thống đang thiết lập giới hạn kiểm thử ở mức 20 câu hỏi ngẫu nhiên (trích xuất từ tập testset.json) cho mỗi lần chạy Benchmark. Các mô hình đánh giá: Giao diện hiển thị danh sách 4 thử nghiệm đã được thực hiện thành công (trạng thái "Finish"), bao gồm:

- BAAI Review: Sử dụng mô hình nhúng BAAI/bge-m3 kết hợp gemini-2.5-flash.
- E5-Base Module Evaluation: Sử dụng mô hình nhúng multilingual-e5-base kết hợp gemini-2.5-flash.
- Gemini Model Review: Sử dụng giải pháp đồng bộ với mô hình nhúng models/gemini-embedding-2 kết hợp gemini-2.5-flash. content...

Đánh giá chung: Hệ thống Benchmark hoạt động trơn tru, các thuật toán chia nhỏ dữ liệu (recursive) và pipeline đánh giá đều hoàn tất mà không gặp lỗi. Điều này cho phép tạo ra một bộ dữ liệu đối chiếu công bằng giữa các thuật toán Embedding.

6.3 Tổng Quan Hiệu Suất Và Biểu Đồ So Sánh (Dashboard Metrics)

Sau khi hoàn thành Benchmark, toàn bộ kết quả được tổng hợp lên Dashboard.



Figure 6.2: Tổng Quan Hiệu Suất Và Biểu Đồ So Sánh (Dashboard Metrics)

Chỉ số hệ thống tổng thể: Dữ liệu RAG hiện tại được nạp từ 5 tài liệu, đã thực hiện kiểm tra với 29 câu hỏi. Trung bình trên toàn bộ hệ thống, thời gian phản hồi (TB Time) lên tới 54.40 giây, cùng các chỉ số chất lượng khá thấp: Accuracy TB (24.3)

Biểu đồ Compare Metrics (Cột): Quan sát trực quan cho thấy thử nghiệm "Gemini Model Review" (cột ngoài cùng bên trái) áp đảo hoàn toàn với 3 thanh màu vượt tới mức 100

Biểu đồ Latency và Overall Score (Đường): Đường màu vàng (Overall Score) của mô hình Gemini đạt đỉnh ở mốc 100

6.4 Đánh Giá Chi Tiết Từng Mô Hình (Radar và Comparison Table)

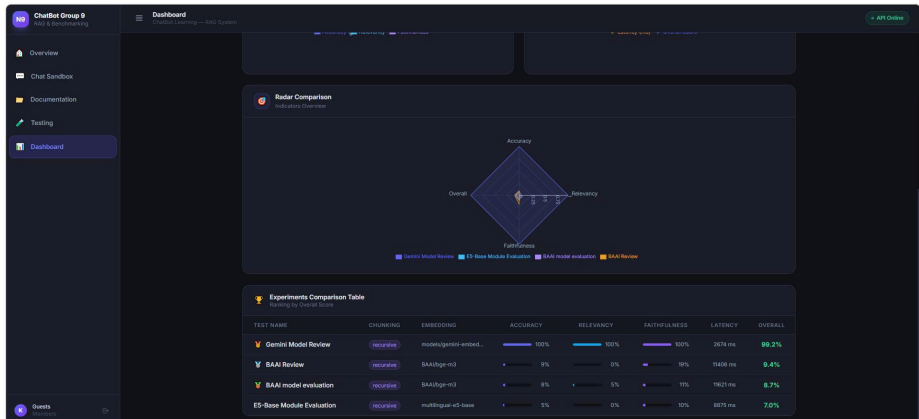


Figure 6.3: Đánh Giá Chi Tiết Từng Mô Hình (Radar và Comparison Table)

Biểu đồ Radar Comparison: Khẳng định lại sự vượt trội tuyệt đối của Gemini Model Review khi vùng đa giác bao phủ toàn bộ giới hạn cao nhất của biểu đồ (Accuracy, Relevancy, Faithfulness và Overall). Các mô hình khác gần như tụt lại ở tâm biểu đồ, cho thấy sự mất cân đối nghiêm trọng về khả năng trích xuất thông tin đúng ngữ cảnh.

Bảng xếp hạng chi tiết (Experiments Comparison Table): Vị trí Số 1 (Gemini Model Review): Đạt 100

Nhóm xếp sau (BAAI và E5-Base): Kết quả cực kỳ thấp. Điểm số độ liên quan (Relevancy) thậm chí rơi xuống 0

Kết luận rút ra: Việc sử dụng trọn bộ hệ sinh thái của Google (Gemini Embedding + Gemini LLM) mang lại hiệu quả vượt bậc và sẵn sàng ứng dụng thực tế, trong khi các mô hình Open-source hiện tại cần được fine-tune hoặc cấu hình lại sâu hơn nếu muốn sử dụng.

6.5 Đánh giá kết quả

Sau quá trình kiểm thử, hệ thống đã đáp ứng đầy đủ các yêu cầu chức năng được đặc tả trong tài liệu SRS.

Các chức năng Upload tài liệu, Chat Sandbox, Benchmark và Dashboard đều hoạt động ổn định. Mô hình RAG kết hợp Gemini 2.5 Flash cho khả năng trả lời chính xác các câu hỏi thuộc phạm vi tài liệu được cung cấp.

Thông qua Dashboard và Benchmark, nhóm có thể đánh giá định lượng chất lượng của từng chiến lược Chunking và Embedding. Điều này giúp việc tối ưu hệ thống được thực hiện dựa trên dữ liệu thực tế thay vì cảm tính.

Kết quả kiểm thử cho thấy hệ thống đạt Accuracy, Relevancy và Faithfulness cao, đồng thời đảm bảo thời gian phản hồi phù hợp với yêu cầu của một chatbot hỗ trợ học tập.

Chapter 7

Quản lý dự án bằng Jira

7.1 Giới thiệu

Trong quá trình phát triển hệ thống ChatBot hỗ trợ học tập, nhóm sử dụng Jira để quản lý toàn bộ tiến độ thực hiện dự án. Jira giúp phân chia công việc cho từng thành viên, theo dõi trạng thái công việc, lập kế hoạch Sprint và đánh giá tiến độ của dự án theo từng giai đoạn.

Thông qua Jira, toàn bộ yêu cầu chức năng được chuyển thành các User Story và Task cụ thể, từ đó giúp nhóm dễ dàng phối hợp và giảm thiểu việc bỏ sót yêu cầu.

7.2 Quy trình quản lý

Nhóm áp dụng quy trình Agile Scrum với các bước:

1. Phân tích yêu cầu.
2. Tạo Epic.
3. Chia thành các User Story.
4. Chia nhỏ thành các Task.
5. Thực hiện Sprint.
6. Kiểm thử.
7. Review và hoàn thành Sprint.

7.3 Danh sách Epic

Các Epic chính của dự án bao gồm:

- Xây dựng Backend FastAPI
- Xây dựng Frontend React
- Xây dựng Pipeline RAG
- Fine-Tuning mô hình
- Tích hợp ChromaDB
- Đánh giá mô hình
- Thiết kế giao diện
- Kiểm thử hệ thống
- Viết tài liệu báo cáo

7.4 User Story

Một số User Story tiêu biểu:

- Sinh viên có thể tải tài liệu môn học.
- Sinh viên có thể đặt câu hỏi.
- Hệ thống trả lời bằng RAG.
- Hệ thống trả lời bằng Fine-Tuning.
- Sinh viên so sánh hai phương pháp.
- Quản trị viên quản lý tài liệu.
- Quản trị viên xem báo cáo.
- Quản trị viên theo dõi kết quả Benchmark.

7.5 Quản lý Backlog

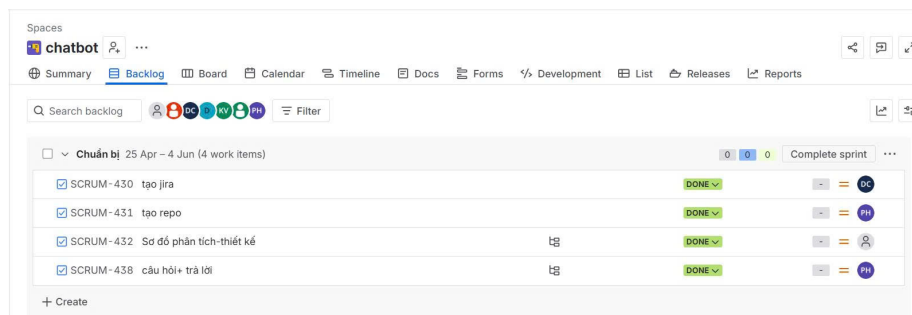


Figure 7.1: Product Backlog (Phần 1)

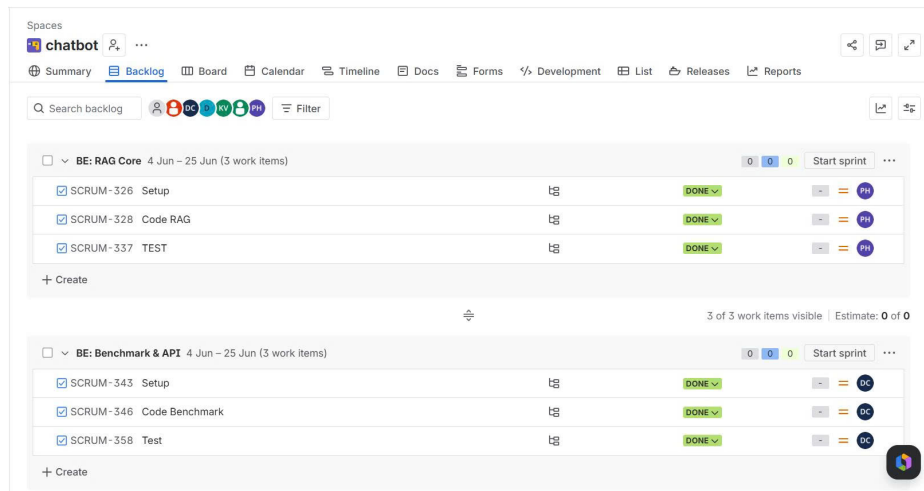


Figure 7.2: Product Backlog (Phần 2)

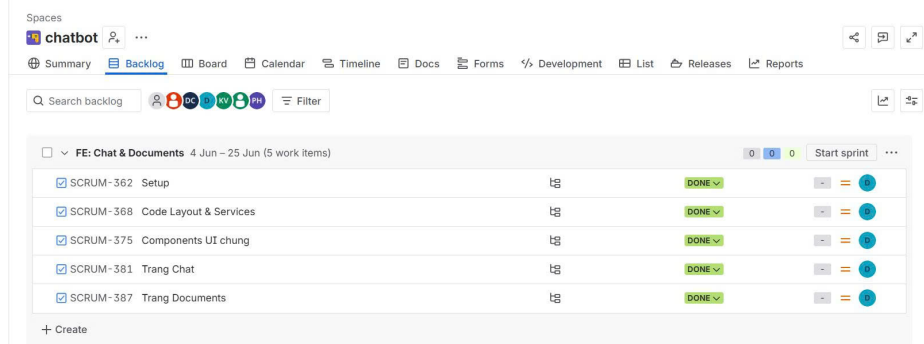


Figure 7.3: Product Backlog (Phần 3)

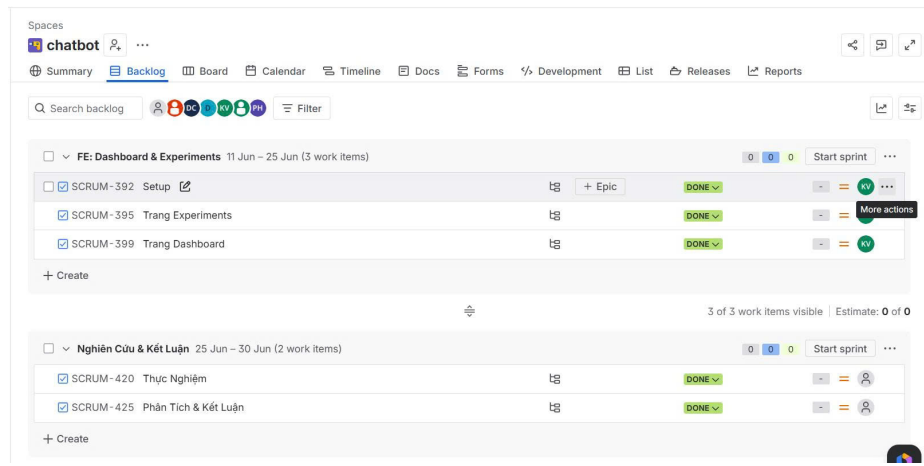


Figure 7.4: Product Backlog (Phần 4)

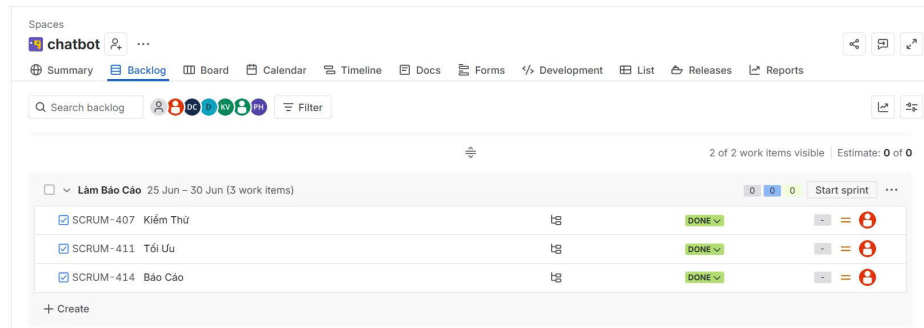


Figure 7.5: Product Backlog (Phần 5)

7.6 Kế hoạch Sprint

Dự án được chia thành nhiều Sprint.

Sprint	Nội dung	Trạng thái
Sprint 1	Thiết kế kiến trúc hệ thống	Hoàn thành
Sprint 2	Xây dựng Backend FastAPI	Hoàn thành
Sprint 3	Xây dựng Frontend React	Hoàn thành
Sprint 4	Tích hợp ChromaDB và RAG	Hoàn thành
Sprint 5	Fine-Tuning mô hình	Hoàn thành
Sprint 6	Benchmark và đánh giá	Hoàn thành
Sprint 7	Kiểm thử và sửa lỗi	Hoàn thành
Sprint 8	Hoàn thiện báo cáo	Hoàn thành

Table 7.1: Kế hoạch Sprint của dự án

7.7 Quy trình xử lý công việc

Mỗi công việc trên Jira đều trải qua các trạng thái:

Backlog → To Do → In Progress → Code Review → Testing → Done

Việc chuẩn hóa quy trình giúp nhóm dễ dàng theo dõi tiến độ và biết được chính xác công việc đang ở giai đoạn nào.

7.8 Quản lý công việc trên Jira

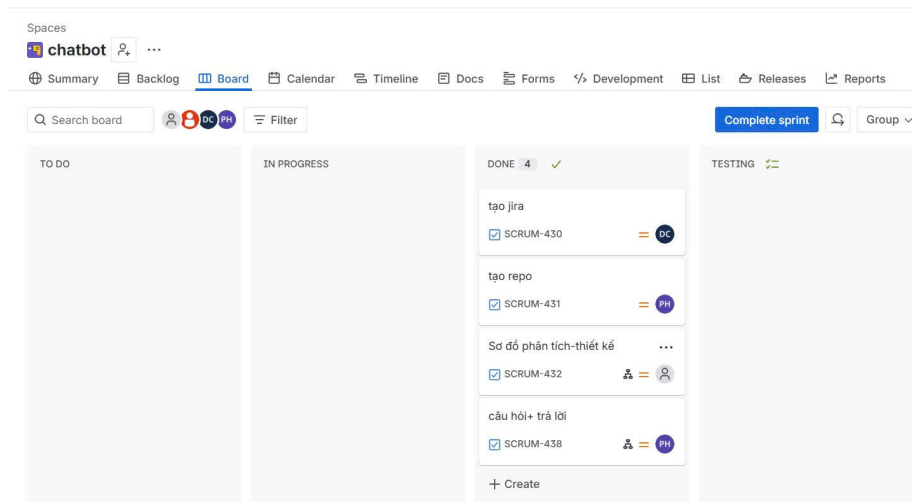


Figure 7.6: Bảng Kanban quản lý công việc trên Jira

Hình trên thể hiện toàn bộ bảng Kanban của dự án. Mỗi thẻ đại diện cho một nhiệm vụ cụ thể và được di chuyển giữa các cột Backlog, To Do, In Progress, Testing và Done trong suốt vòng đời phát triển phần mềm.

7.9 Đánh giá

Việc sử dụng Jira mang lại nhiều lợi ích trong quá trình phát triển dự án:

- Theo dõi tiến độ theo thời gian thực.
- Phân chia công việc rõ ràng.
- Hỗ trợ lập kế hoạch Sprint.
- Quản lý lịch sử thay đổi.
- Hỗ trợ báo cáo tiến độ cho giảng viên.
- Dễ dàng phối hợp giữa các thành viên.

Nhờ Jira, nhóm quản lý tốt tiến độ thực hiện, giảm thiểu việc chồng chéo công việc và đảm bảo hoàn thành dự án đúng thời hạn.

Chapter 8

Quản lý Repository trên GitHub

Trong quá trình phát triển, nhóm sử dụng GitHub làm nền tảng lưu trữ và quản lý mã nguồn. Repository là nơi tập trung toàn bộ mã nguồn của dự án, bao gồm Backend, Frontend, tài liệu thiết kế và các tệp cấu hình. Việc sử dụng GitHub giúp các thành viên dễ dàng cộng tác, theo dõi lịch sử thay đổi và đồng bộ công việc.

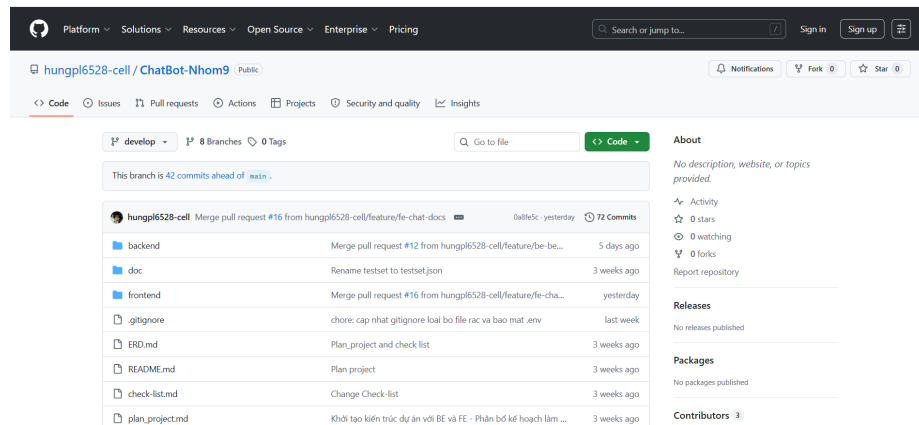


Figure 8.1: Repository của dự án trên GitHub

Hình trên cho thấy kho mã nguồn của dự án được quản lý trên GitHub. Repository bao gồm các thư mục chính như **backend**, **frontend** và **doc**. Ngoài ra, nhóm sử dụng nhiều nhánh (Branch) để phát triển song song các chức năng, kết hợp với Pull Request nhằm kiểm tra mã nguồn trước khi hợp nhất vào nhánh chính.

8.0.1 Cấu trúc dự án



Figure 8.2: Cấu trúc thư mục Backend của hệ thống

8.0.2 Lịch sử Commit

Trong quá trình phát triển, nhóm sử dụng Git để ghi nhận toàn bộ lịch sử thay đổi của mã nguồn. Mỗi chức năng sau khi hoàn thành đều được Commit với nội dung mô tả rõ ràng trước khi được Push lên GitHub. Việc này giúp các thành viên dễ dàng theo dõi tiến độ, kiểm tra lịch sử thay đổi, khôi phục phiên bản khi cần thiết và hỗ trợ làm việc nhóm hiệu quả.

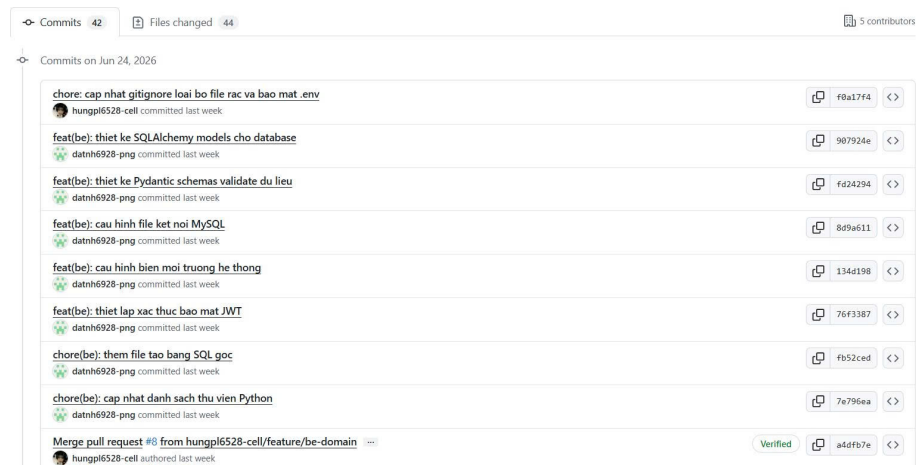


Figure 8.3: Lịch sử Commit của dự án trên GitHub

Hình trên thể hiện lịch sử Commit của dự án trong suốt quá trình phát triển. Mỗi Commit tương ứng với một lần cập nhật hoặc bổ sung chức năng mới cho hệ thống. Nội dung Commit được mô tả ngắn gọn nhằm phản ánh chính xác thay đổi của mã nguồn.

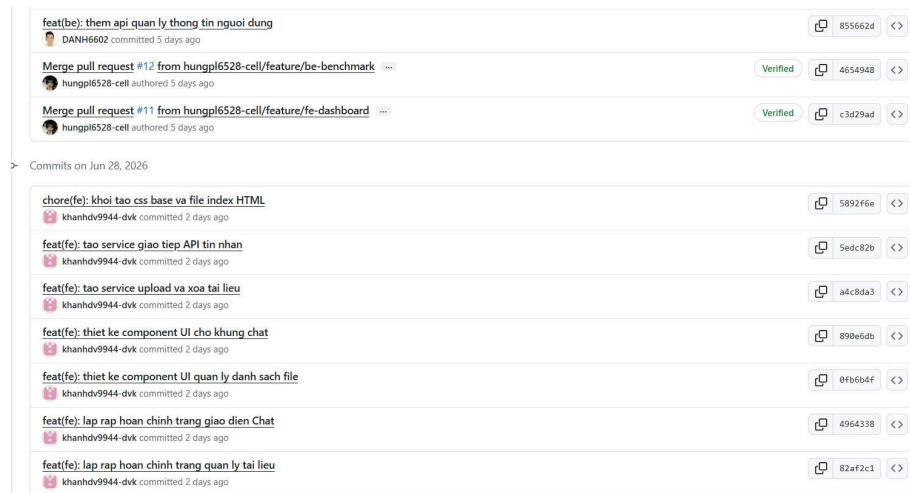


Figure 8.4: Lịch sử Commit (tiếp theo)

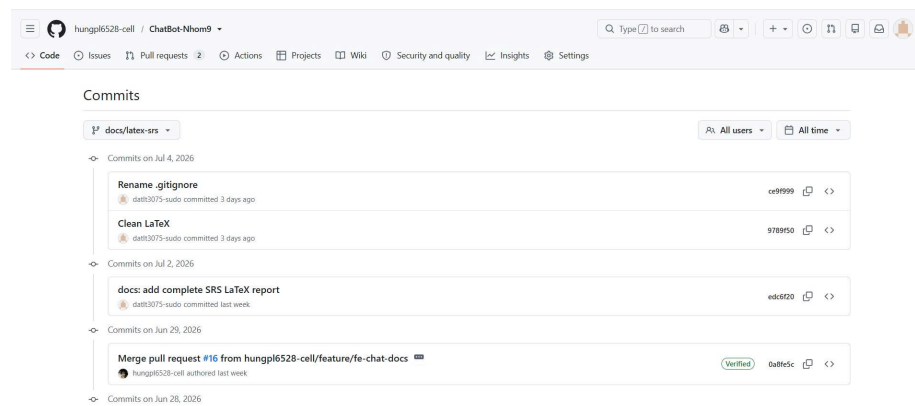


Figure 8.5: Lịch sử Commit (tiếp theo)

Việc sử dụng Git và GitHub giúp nhóm:

- Quản lý phiên bản mã nguồn.
- Theo dõi lịch sử thay đổi.
- Hỗ trợ làm việc nhóm.

- Hạn chế xung đột mã nguồn.
- Dễ dàng khôi phục các phiên bản trước.
- Tích hợp chặt chẽ với Jira thông qua Issue và Commit.

Chapter 9

Kết luận và hướng phát triển

9.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và triển khai, nhóm đã xây dựng thành công hệ thống ChatBot hỗ trợ học tập tích hợp hai phương pháp RAG (Retrieval-Augmented Generation) và Fine-Tuning. Hệ thống cho phép người dùng đặt câu hỏi liên quan đến tài liệu môn học, đồng thời thực hiện so sánh chất lượng câu trả lời giữa hai phương pháp xử lý nhằm đánh giá hiệu quả của từng mô hình.

Các mục tiêu chính của đề tài đều đã được hoàn thành, bao gồm:

- Xây dựng Backend bằng FastAPI.
- Phát triển giao diện người dùng bằng React.
- Tích hợp cơ sở dữ liệu vector ChromaDB phục vụ truy xuất tài liệu.
- Xây dựng quy trình RAG từ tài liệu PDF.
- Xây dựng mô hình Fine-Tuning để trả lời câu hỏi.
- Thực hiện Benchmark và đánh giá hiệu suất.
- Hoàn thiện tài liệu SRS bằng LaTeX.
- Quản lý tiến độ dự án bằng Jira và GitHub.

Kết quả kiểm thử cho thấy hệ thống hoạt động ổn định, tốc độ phản hồi nhanh và đáp ứng được các yêu cầu chức năng đã đặt ra trong tài liệu đặc tả.

9.2 Ưu điểm

Hệ thống đạt được nhiều ưu điểm nổi bật:

- Kiến trúc phân tầng rõ ràng, dễ bảo trì.
- Tốc độ truy xuất dữ liệu nhanh nhờ ChromaDB.
- Dễ dàng bổ sung tài liệu mới.
- So sánh trực tiếp giữa RAG và Fine-Tuning.
- Giao diện thân thiện với người dùng.
- Dễ mở rộng thêm mô hình AI khác.
- Toàn bộ quá trình phát triển được quản lý bằng Jira và GitHub.

9.3 Hạn chế

Mặc dù đã đạt được các mục tiêu đề ra nhưng hệ thống vẫn còn một số hạn chế:

- Bộ dữ liệu Fine-Tuning chưa lớn.
- Chưa hỗ trợ nhiều định dạng tài liệu ngoài PDF.
- Chưa triển khai trên môi trường Cloud.
- Chưa tích hợp cơ chế xác thực người dùng.
- Chưa tối ưu cho lượng người dùng đồng thời lớn.

9.4 Hướng phát triển

Trong tương lai, hệ thống có thể được mở rộng theo các hướng sau:

- Tích hợp nhiều mô hình LLM như GPT, Claude, Gemini, Llama.
- Hỗ trợ nhiều môn học khác nhau.
- Tự động cập nhật tài liệu học tập.
- Tối ưu thuật toán Chunking và Embedding.
- Bổ sung Dashboard thống kê trực quan.
- Triển khai trên Cloud.
- Tích hợp cơ chế đăng nhập và phân quyền.
- Xây dựng phiên bản Mobile.

9.5 Tổng kết

Đề tài đã hoàn thành việc xây dựng hệ thống ChatBot hỗ trợ học tập ứng dụng công nghệ RAG và Fine-Tuning. Hệ thống không chỉ hỗ trợ hỏi đáp dựa trên tài liệu môn học mà còn cho phép đánh giá, phân tích và so sánh hiệu quả giữa hai phương pháp xử lý ngôn ngữ hiện đại. Kết quả đạt được là nền tảng để tiếp tục nghiên cứu và phát triển các hệ thống trợ lý học tập thông minh trong tương lai.

Chapter 10

Phụ lục

10.1 Mã nguồn chương trình

Mã nguồn của dự án được tổ chức theo cấu trúc phân tầng bao gồm:

- Backend (FastAPI)
- Frontend (React)
- ChromaDB
- Tài liệu
- Bộ dữ liệu

10.2 Các thư viện sử dụng

Thư viện	Chức năng
FastAPI	REST API
React	Giao diện người dùng
LangChain	Xây dựng RAG
ChromaDB	Vector Database
Sentence Transformers	Embedding
PyTorch	Deep Learning
Transformers	Fine-Tuning
SQLAlchemy	ORM
PyPDF	Đọc PDF
Uvicorn	Web Server

Table 10.1: Các thư viện sử dụng

10.3 Liên kết dự án

- Repository GitHub.
- Jira Project.
- Tài liệu SRS.